Practice Worksheet 3: Functions

CORE 103 Computational Thinking

Fall 2026

Instructions

- This PW is **UNGRADED**. However, it is highly encouraged that you attempt and submit it to get feedback from your section tutor.
- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW3_aa123456.zip` where `aa123456` will be replaced with your student id.

Exercises

1. Campus Gum Share

This question should be implemented in the file ***q1.py****.*

Areeba is attending a short summer program abroad with a few university friends. Before a long lab session, she buys some packs of gum from the campus store. Each pack costs \$1.50. Since everyone will share the gum, the group agrees that the total cost should be split equally among all people who will eat it, including Areeba herself.

Areeba does not want to calculate the share manually every time the group size changes. Given the number of gum packs `gum_packs` and the number of friends `friend_count`, help her find how much each person should pay. The value `friend_count` counts Areeba's friends only, so Areeba is one additional person sharing the cost. All provided inputs allow the total cost to be divided exactly into whole cents.

Note: 1 dollar = 100 cents. Therefore, \$1.50 = 150 cents. You may perform the calculations in cents and then separate each person's share into dollars and cents.

Task

Write a function, `split_gum_cost`, that takes `gum_packs` and `friend_count` as parameters.

The function must calculate each person's share and print it in dollars and cents.

After defining the function, read the two input values and call the function.

Constraints

- `gum_packs` is an integer.
- `friend_count` is an integer.
- `1 <= gum_packs <= 99`

- `1 <= friend_count <=10`

Interaction

The input contains two lines:

- The first line contains `gum_packs`.
- The second line contains `friend_count`.

The program must print a single line showing each person's share in dollars and cents, as shown in the Sample section.

Sample

Input	Output
5 2	Each person spends 2 dollars and 50 cents
3 1	Each person spends 2 dollars and 25 cents

In the first case, each pack costs \$1.50 = 150 cents, so 5 packs cost 750 cents. Including Areeba and her 2 friends, there are 3 people sharing the cost. Each person's share is 750 / 3 = 250 cents. Since 1 dollar = 100 cents, 250 cents is 2 dollars and 50 cents.

In the second case, 3 packs cost 450 cents. Including Areeba and her 1 friend, there are 2 people sharing the cost. Each person's share is 450 / 2 = 225 cents, which is 2 dollars and 25 cents.

Python Implementation

Implement the Python code in **q1.py**.

2. Study Room AC Truce

*This question should be implemented in the file **q2.py**.*

Three friends are studying together in the university lounge and need to agree on one air-conditioner setting. Ali prefers warmer rooms and wants the room to be at least `ali_min_temp` degrees Celsius. Raahim gets uncomfortable if the room is too warm, so he wants it to be at most `raahim_max_temp` degrees Celsius. Saad also feels cold easily and wants the room to be at least `saad_min_temp` degrees Celsius.

Task

Write a function, `can_agree_on_temperature`, that takes the following parameters:

- `ali_min_temp`
- `raahim_max_temp`
- `saad_min_temp`

The function must return `"YES"` if there is at least one temperature setting that satisfies all three friends, and `"NO"` if no such setting is possible.

After defining the function, read the three input values, call the function, and print the returned value.

Constraints

- `16 <= ali_min_temp <=30`
- `16 <= raahim_max_temp <=30`

- `16 <= saad_min_temp <=30`

Interaction

The input contains three lines:

- The first line contains `ali_min_temp`.
- The second line contains `raahim_max_temp`.
- The third line contains `saad_min_temp`.

The program must print either `"YES"` or `"NO"`.

Sample

Input	Output
22 25 20	YES
24 22 23	NO
24 24 20	YES

In the first case, `ali_min_temp =22`, `raahim_max_temp =25`, and `saad_min_temp =20`. Ali wants the temperature to be at least 22, Raahim wants it to be at most 25, and Saad wants it to be at least 20. Temperatures from 22 to 25 satisfy their preferences, so the answer is `"YES"`.

In the second case, Ali wants the temperature to be at least 24, while Raahim wants it to be at most 22. These requirements cannot both be satisfied, so the answer is `"NO"`.

In the third case, a setting of exactly 24 degrees satisfies all three friends. This shows that the words *at least* and *at most* include the boundary value.

Python Implementation

Implement the Python code in **q2.py**.

3. Weekend Trip Car Choice

*This question should be implemented in the file **q3.py**.*

A group of students is planning a weekend trip. The destination is `trip_distance_km` kilometers away, and they can rent either a petrol car or a diesel car.

One litre of petrol costs `petrol_price` rupees, while one litre of diesel costs `diesel_price` rupees.

The petrol car travels `petrol_km_per_litre` kilometers per litre.

The diesel car travels `diesel_km_per_litre` kilometers per litre.

Fuel can be bought in any amount, not necessarily an integer number of litres.

For either car, the number of litres needed for the trip is the trip distance divided by that car's kilometers-per-litre value.

Task

Write a function, `ride_choice`, that takes the following parameters:

- `trip_distance_km`

- `petrol_price`
- `diesel_price`
- `petrol_km_per_litre`
- `diesel_km_per_litre`

The function should calculate the fuel cost for the one-way trip to the destination for both cars and compare the two costs.

The function must return:

- `"Diesel"` if the diesel car is cheaper,
- `"Petrol"` if the petrol car is cheaper, and
- `"Any"` if both cars cost the same.

After defining the function, read the five input values, call the function, and print the returned value.

Constraints

- All five input values are integers.
- `1 <= trip_distance_km <=1000`
- `1 <= petrol_price <= 1000`
- `1 <= diesel_price <= 1000`
- `1 <= petrol_km_per_litre <=100`
- `1 <= diesel_km_per_litre <=100`
- Test cases where both calculated trip costs are equal are chosen so that using `==` works correctly.

Interaction

The input contains five lines:

- The first line contains `trip_distance_km`, the one-way distance to the destination.
- The second line contains `petrol_price`, the price of one litre of petrol.
- The third line contains `diesel_price`, the price of one litre of diesel.
- The fourth line contains `petrol_km_per_litre`, the distance traveled per litre by the petrol car.
- The fifth line contains `diesel_km_per_litre`, the distance traveled per litre by the diesel car.

The program must print exactly one of `"Petrol"`, `"Diesel"`, or `"Any"`.

Sample

Input	Output
120 290 280 12 16	Diesel
150 285 300 15 12	Petrol
100 300 240 10 8	Any

In the first case, the petrol car requires 10 litres, costing 2900 rupees, while the diesel car requires 7.5 litres, costing 2100 rupees. Therefore, the diesel car is cheaper, so the function returns `"Diesel"`.

In the second case, the petrol car requires 10 litres, costing 2850 rupees, while the diesel car requires 12.5 litres, costing 3750 rupees. Therefore, the petrol car is cheaper, so the function returns `"Petrol"`.

In the third case, the petrol car requires 10 litres and costs 3000 rupees. The diesel car requires 12.5 litres and also costs 3000 rupees. Since both cars cost the same, the function returns `"Any"`.

Python Implementation

Implement the Python code in **q3.py**.

4. Family Trip Buffet Budget

*This question should be implemented in the file **q4.py**.*

Areeba's family is staying at a hill resort for a weekend trip. Breakfast, lunch, and dinner buffets are charged separately for each guest.

The resort charges different buffet prices for adults, children, and toddlers. Since every family member has different food plans for the day, Areeba wants a small function that can calculate the daily buffet bill for any one person.

The buffet charges are as follows:

	Breakfast	Lunch	Dinner
Adult (ages 13 and above)	Rs. 800	Rs. 1200	Rs. 1500
Child (ages 6 to 12)	Rs. 400	Rs. 700	Rs. 800
Toddler (ages 5 and below)	FREE	FREE	Rs. 300

For one family member, the input gives their `age` and three buffet choices:

- `breakfast_choice`
- `lunch_choice`
- `dinner_choice`

Each choice is `'Y'` or `'y'` if the person wants that buffet, and `'N'` or `'n'` otherwise.

Task

Write a function, `daily_buffet_cost`, that takes the following parameters:

- `age`
- `breakfast_choice`
- `lunch_choice`
- `dinner_choice`

The function must calculate the total daily buffet charges for that person and print the appropriate message using `Adult`, `Child`, or `Toddler`, according to the person's age.

After defining the function, read the four input values and call the function.

Constraints

- `age` is an integer.
- `age > 0`
- `breakfast_choice`, `lunch_choice`, and `dinner_choice` are each one of `'Y'`, `'y'`, `'N'`, or `'n'`.

Interaction

The input contains four lines:

- The first line contains `age`.
- The second line contains `breakfast_choice`.
- The third line contains `lunch_choice`.
- The fourth line contains `dinner_choice`.

The program must print one statement giving the total daily buffet charges for the guest, using `Adult`, `Child`, or `Toddler` according to the person's age, as shown in the Sample section.

Sample

Input	Output
28 Y y N	Adult charges for the day will be: Rs. 2000
10 y n N	Child charges for the day will be: Rs. 400
5 y y y	Toddler charges for the day will be: Rs. 300

In the first case, the guest is 28 years old, so the guest is an adult. The guest selects breakfast and lunch only. The total charges are Rs. 800 + Rs. 1200 = Rs. 2000.

In the second case, the guest is 10 years old, so the guest is a child. The guest selects only breakfast. The total charges are Rs. 400.

In the third case, the guest is 5 years old, so the guest is a toddler. Breakfast and lunch are free for toddlers, while dinner costs Rs. 300. Therefore, the total charges are Rs. 300.

Python Implementation

Implement the Python code in **q4.py**.

5. Cafe Hand Game

This question should be implemented in the file ***q5.py****.*

Three university friends, Areeba, Bilal, and Zoya, are sitting in the cafeteria deciding who gets to choose the next table game. They settle it with a quick round of *pugam pugai*.

In one round, all three players show their hands at the same time. Each player can show either `'UP'` or `'DOWN'`. The player whose hand is different from the other two wins the round. If all three players show the same hand, the result is a draw.

Given `areeba_hand`, `bilal_hand`, and `zoya_hand`, determine who wins the round.

Task

Write a function, `pugam_pugai`, that takes `areeba_hand`, `bilal_hand`, and `zoya_hand` as parameters.

The function must return the winner's name: `"Areeba"`, `"Bilal"`, or `"Zoya"`. If no one wins, return `"Draw"`.

After defining the function, read the three input values, call the function, and print the returned value.

Constraints

- `areeba_hand`, `bilal_hand`, and `zoya_hand` are strings.
- Each of `areeba_hand`, `bilal_hand`, and `zoya_hand` is either `'UP'` or `'DOWN'`.

Interaction

The input contains three lines:

- The first line contains `areeba_hand`.
- The second line contains `bilal_hand`.
- The third line contains `zoya_hand`.

The program must print exactly one of `"Areeba"`, `"Bilal"`, `"Zoya"`, or `"Draw"`.

Sample

Input	Output
DOWN UP UP	Areeba
DOWN UP DOWN	Bilal
UP UP DOWN	Zoya
UP UP UP	Draw

In the first case, Areeba shows `'DOWN'` while both Bilal and Zoya show `'UP'`, so Areeba has the unique hand and wins.

In the second case, Bilal shows `'UP'` while both Areeba and Zoya show `'DOWN'`, so Bilal has the unique hand and wins.

In the third case, Zoya shows `'DOWN'` while both Areeba and Bilal show `'UP'`, so Zoya has the unique hand and wins.

In the fourth case, all three friends show `'UP'`, so the result is a draw.

Python Implementation

Implement the Python code in **q5.py**.

6. Campus Reimbursement Counter

*This question should be implemented in the file **q6.py**.*

During a university food festival, students spend money on decorations, printing, snacks, and stall materials. At the end of the event, the finance team reimburses them in cash.

For this exercise, assume that the reimbursement counter uses only the following denominations:

Rs.1000, Rs.500, Rs.100, Rs.10, and Rs.1

To make the process quick, the counter always uses the highest possible denomination first and then uses smaller denominations for the remaining amount.

Task

Write a function, `money_changer`, that takes `reimbursement_amount` as a parameter.

The function must print the number of notes or coins required for each denomination, from highest to lowest.

After defining the function, read `reimbursement_amount` and call the function.

Constraints

- `reimbursement_amount` is an integer.
- `reimbursement_amount >=1`

Interaction

The input contains one line with the integer `reimbursement_amount`, representing the amount to be reimbursed in rupees.

The program must print five lines showing the quantity required for each denomination, from highest to lowest, as shown in the Sample section.

Sample

Input	Output
10200	1000 : 10 500 : 0 100 : 2 10 : 0 1 : 0
2769	1000 : 2 500 : 1 100 : 2 10 : 6 1 : 9

In the first case, the reimbursement amount is Rs.10200. The counter gives 10 notes of Rs.1000, leaving Rs.200. It then gives 2 notes of Rs.100, leaving no remaining amount.

In the second case, the reimbursement amount is Rs.2769. The counter gives 2 notes of Rs.1000, 1 note of Rs.500, 2 notes of Rs.100, 6 notes or coins of Rs.10, and 9 coins of Rs.1.

Python Implementation

Implement the Python code in **q6.py**.

7. Exchange Student Clock Helper

*This question should be implemented in the file **q7.py**.*

An exchange student has just arrived on campus and keeps mixing up the 24-hour clock used in schedules with the 12-hour format they are used to. They have already missed one society meeting because 13:05 looked confusing.

To help, you are asked to create a function that converts a given 24-hour time into the corresponding 12-hour AM/PM format.

Task

Write a function, `time_conversion`, that takes `hour_24` and `minute` as parameters.

If the given time is valid, the function must return the corresponding 12-hour time as a string followed by `"AM"` or `"PM"`.

The minute part must always contain two digits. For example, 5 minutes should appear as `05`.

Using a Built-in Function

For this question, you may use Python's `str()` function and string concatenation with `+` when building the returned time string. For example:

```
>>> minute = 5
>>> "0" + str(minute)
'05'
>>> hour_12 = 1
>>> str(hour_12) + ":05 PM"
'1:05 PM'
```

If either `hour_24` or `minute` is outside the valid range, the function must return `"Invalid Time"`.

After defining the function, read the two input values, call the function, and print the returned value.

Constraints

- `hour_24` is an integer.
- `minute` is an integer.
- The input values may represent either a valid or an invalid time.

A valid 24-hour time satisfies:

- `0 <= hour_24 <= 23`
- `0 <= minute <= 59`

Interaction

The input contains two lines:

- The first line contains `hour_24`.

- The second line contains `minute`.

The program must print the converted time in 12-hour AM/PM format if the input is valid, or `"Invalid Time"` otherwise.

Sample

Input	Output
13 5	1:05 PM
12 61	Invalid Time
0 23	12:23 AM
12 0	12:00 PM

In the first case, 13:05 in 24-hour format corresponds to 1:05 PM.

In the second case, 61 is not a valid value for minutes, so the function returns `"Invalid Time"`.

In the third case, 00:23 corresponds to 12:23 AM.

In the fourth case, 12:00 corresponds to 12:00 PM.

Python Implementation

Implement the Python code in **q7.py**.

8. Semester GPA Calculator

*This question should be implemented in the file **q8.py**.*

At the end of the semester, students want to calculate their grades and GPA based on their marks in four courses.

The university uses the following grading scheme:

Marks	Grade	Grade Point
$[95, 100]$	A+	4.00
$[90, 95)$	A	4.00
$[85, 90)$	A-	3.67
$[80, 85)$	B+	3.33
$[75, 80)$	B	3.00
$[70, 75)$	B-	2.67
$[67, 70)$	C+	2.33
$[63, 67)$	C	2.00
$[60, 63)$	C-	1.67
$[0, 60)$	F	0.00

For example, $[90, 95)$ means that the mark is at least 90 but less than 95.

All four courses have the same number of credit hours. Therefore, the GPA is the arithmetic mean of the four grade points.

Task

Write a main function, `gpa_calculator`, that takes `course_1_mark`, `course_2_mark`, `course_3_mark`, and `course_4_mark` as parameters.

The function must print the grades of all four courses followed by the student's GPA.

Your solution must also use the following two helper functions:

- `get_letter_grade(mark)`, which returns the letter grade corresponding to `mark`.
- `get_grade_point(letter_grade)`, which returns the grade point corresponding to `letter_grade`.

The `gpa_calculator` function must use these helper functions when determining the grades and grade points.

Round the GPA to **at most 2 decimal places** before printing it.

Using a Built-in Function

For this question, you may use Python's built-in `round()` function to round the GPA. For example:

```
>>> semester_gpa = 3.4567
>>> round(semester_gpa, 2)
3.46
>>> semester_gpa = 3.0
>>> round(semester_gpa, 2)
3.0
```

The second argument specifies the maximum number of decimal places to which the value is rounded.

After defining all three functions, read the four input values and call `gpa_calculator`.

Constraints

- `course_1_mark`, `course_2_mark`, `course_3_mark`, and `course_4_mark` are integers.
- `0 <= course_1_mark <=100`
- `0 <= course_2_mark <=100`
- `0 <= course_3_mark <=100`
- `0 <= course_4_mark <=100`

Interaction

The input contains four lines:

- The first line contains `course_1_mark`.
- The second line contains `course_2_mark`.
- The third line contains `course_3_mark`.
- The fourth line contains `course_4_mark`.

The program must print two lines. The first line displays the four letter grades, and the second line displays the GPA, as shown in the Sample section.

Sample

Input	Output
100 77 64 30	Your grades are A+ B C F Your GPA is 2.25
75 91 66 50	Your grades are B A C F Your GPA is 2.25

In the first case, the grades are `A+`, `B`, `C`, and `F`, with grade points `4.00`, `3.00`, `2.00`, and `0.00`. Their average is `2.25`.

In the second case, the grades are `B`, `A`, `C`, and `F`, with grade points `3.00`, `4.00`, `2.00`, and `0.00`. Their average is also `2.25`.

Python Implementation

Implement the Python code in **q8.py**.

Rough Work